Connect to remote agents with A2A protocol

In this exercise, you'll use Azure AI Agent Service with the A2A protocol to create simple remote agents that interact with one another. These agents will assist technical writers with preparing their developer blog posts. A title agent will generate a headline, and an outline agent will use the title to develop a concise outline for the article. Let's get started.

> **Tip**: The code used in this exercise is based on the Microsoft Foundry SDK for Python. You can develop similar solutions using the SDKs for Microsoft .NET, JavaScript, and Java. Refer to Microsoft Foundry SDK client libraries for details.

This exercise should take approximately **30** minutes to complete.

> **Note**: Some of the technologies used in this exercise are in preview or in active development. You may experience some unexpected behavior, warnings, or errors.

Prerequisites

Before starting this exercise, ensure you have:

- Visual Studio Code installed on your local machine
- An active Azure subscription
- Python 3.13 installed
- Git installed on your local machine

> * Python 3.14 isn't supported yet: some dependencies have no 3.14 build. This lab was tested with Python 3.13.12.

Create a Foundry project with the Foundry Toolkit for VS Code extension

As a developer, you may spend some time working in the Foundry portal; but you're also likely to spend a lot of time in Visual Studio Code. The Foundry Toolkit for VS Code extension provides a convenient way to work with Foundry project resources without leaving the development environment.

1. Open Visual Studio Code.
2. Select **Extensions** from the left pane (or press **Ctrl+Shift+X**).
3. Search the extensions marketplace for the `Foundry Toolkit` extension from Microsoft and select **Install**.

   > **Note**: The extension is currently listed as **Foundry Toolkit**, but some VS Code labels, commands, or older screenshots may still refer to **AI Toolkit**. In this lab, treat those names as referring to the same extension experience.

4. After installing the extension, select its icon in the sidebar to open the Foundry Toolkit view.

   You should be prompted to sign in to your Azure account if you haven't already.

5. Select **Create Project** under **Microsoft Foundry Resources**.

   If a default project is already active, the project name will appear under **My Resources**. You can create a new project by right-clicking on the active project and selecting **Switch Default Project in Azure Extension**.

6. Select your Azure subscription and resource group, then enter a name for your Foundry project to create a new project for this exercise.

When the deployment is complete, you should see the project appear in the Foundry Toolkit pane as the default project.

Deploy a model

At the core of any generative AI project, there's at least one generative AI model. In this task, you'll deploy a model from the Model Catalog to use with your agent.

1. When the "Project deployed successfully" popup appears, select the **Deploy a new model** button. This opens the Model Catalog.

   > **Tip**: You can also access the Model Catalog by selecting the **+** icon next to **Models** in the Resources section, or by pressing **F1** and running the command **Foundry Toolkit: Show model catalog**.

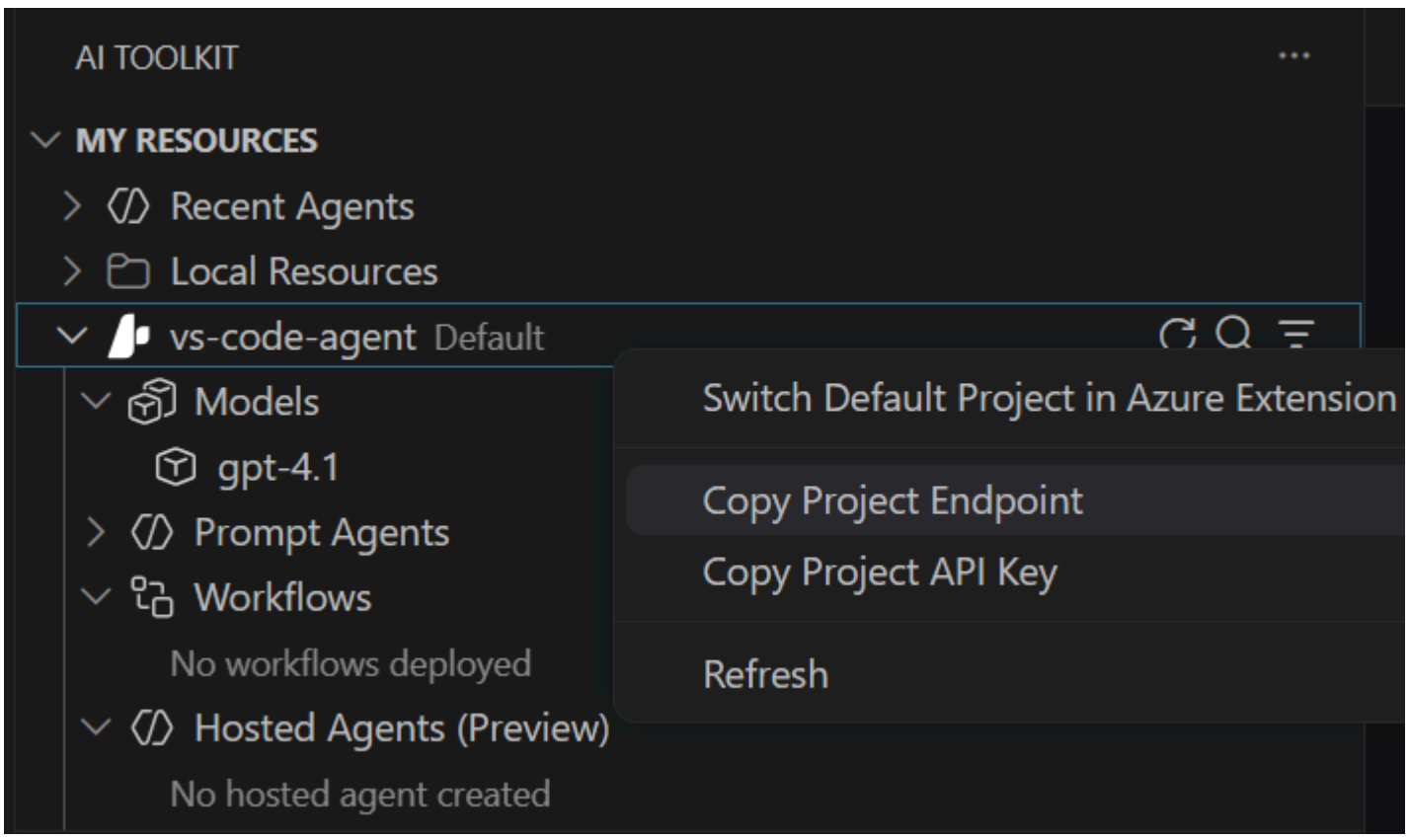
2. In the Model Catalog, locate the **gpt-5** model (you can use the search bar to find it quickly).

3. Select **Deploy** next to the gpt-5 model.
4. Configure the deployment settings:
   - **Deployment name**: Enter a name like "gpt-5"
   - **Deployment type**: Select **Global Standard** (or **Standard** if Global Standard is not available)
   - **Model version**: Leave as default
   - **Tokens per minute**: Leave as default

5. Select **Deploy to Microsoft Foundry** in the bottom-left corner.

6. Wait for the deployment to complete. Your deployed model will appear under the **Models** section in the Resources view.

7. Right-click the name of the project deployment and select **Copy Project Endpoint**. You'll need this URL to connect your agent to the Foundry project in the next steps.



Clone the starter code repository

For this exercise, you'll use starter code that will help you connect to your Foundry project and create an agent that can process expenses data. You'll clone this code from a GitHub repository.

1. In VS Code, open the Command Palette (**Ctrl+Shift+P** or **View > Command Palette**).

2. Type **Git: Clone** and select it from the list.

3. Enter the repository URL:

```
Code                                                    Copy
```

```
https://github.com/MicrosoftLearning/mslearn-ai-agents.git
```

4. Choose a location on your local machine to clone the repository.
5. When prompted, select **Open** to open the cloned repository in VS Code.
6. Once the repository opens, select **File > Open Folder** and navigate to `mslearn-ai-agents/Labfiles/09-build-remote-agents-with-a2a`, then choose **Select Folder**.
7. In the Explorer pane, expand the **Python** folder to view the code files for this exercise.
8. In the Explorer view, navigate to the **Labfiles/09-build-remote-agents-with-a2a/Python** folder to find the starter code for this exercise.

   The provided files include:

Prerequisites
Create a Foundry project with the Foundry Toolkit for VS Code extension
Deploy a model
Clone the starter code repository
Create a discoverable agent
Enable messages between the agents
Test the application
Clean up

Code Copy

```
python
├── outline_agent/
│   ├── agent.py
│   ├── agent_executor.py
│   └── server.py
├── routing_agent/
│   ├── agent.py
│   └── server.py
├── title_agent/
│   ├── agent.py
│   ├── agent_executor.py
│   └── server.py
├── client.py
└── run_all.py
```

   Each agent folder contains the Azure AI agent code and a server to host the agent. The **routing agent** is responsible for discovering and communicating with the **title** and **outline** agents. The **client** allows users to submit prompts to the routing agent. `run_all.py` launches all the servers and runs the client.

9. Right-click on the **requirements.txt** file and select **Open in Integrated Terminal**.
10. In the terminal, enter the following command to install the required Python packages in a virtual environment:

Code Copy

```
python -m venv labenv
.\labenv\Scripts\Activate.ps1
pip install -r requirements.txt
```

11. Open the **.env** file, replace the **your_project_endpoint** placeholder with the endpoint for your project (copied from the project deployment resource in the Foundry Toolkit extension) and ensure that the MODEL_DEPLOYMENT_NAME variable is set to your model deployment name. Use **Ctrl+S** to save the file after making these changes.

Create a discoverable agent

In this task, you create the title agent that helps writers create trendy headlines for their articles. You also define the agent's skills and card required by the A2A protocol to make the agent discoverable.

> **Tip**: As you add code, be sure to maintain the correct indentation. Use the existing comments as a guide, entering the new code at the same level of indentation.

1. Open the **title_agent/agent.py** file in the code editor.
2. Find the comment **Create the agents client** and add the following code to connect to the Azure AI project:

> **Tip**: Be careful to maintain the correct indentation level.

Code Copy

```
# Create the agents client
self.client = AgentsClient(
    endpoint=os.environ['PROJECT_ENDPOINT'],
    credential=DefaultAzureCredential(
        exclude_environment_credential=True,
        exclude_managed_identity_credential=True
    )
)
```

3. Find the comment **Create the title agent** and add the following code to create the agent:

Code Copy

```
# Create the title agent
self.agent = self.client.create_agent(
    model=os.environ['MODEL_DEPLOYMENT_NAME'],
    name='title-agent',
    instructions="""
    You are a helpful writing assistant.
    Given a topic the user wants to write about, suggest a single clear and catchy blog post
title.
    """,
)
```

4. Find the comment **Create a thread for the chat session** and add the following code to create the chat thread:

Code Copy

```
# Create a thread for the chat session
thread = self.client.threads.create()
```

5. Locate the comment **Send user message** and add this code to submit the user's prompt:

Code Copy

```
# Send user message
self.client.messages.create(thread_id=thread.id, role=MessageRole.USER,
content=user_message)
```

6. Under the comment **Create and run the agent**, add the following code to initiate the agent's response generation:

Code Copy

```
# Create and run the agent
run = self.client.runs.create_and_process(thread_id=thread.id, agent_id=self.agent.id)
```

The code provided in the rest of the file will process and return the agent's response.

7. Save the code file (*CTRL+S*). Now you're ready to share the agent's skills and card with the A2A protocol.
8. Open the **title_agent/server.py** file in the code editor.
9. Find the comment **Define agent skills** and add the following code to specify the agent's functionality:

Code Copy

```
# Define agent skills
skills = [
    AgentSkill(
        id='generate_blog_title',
        name='Generate Blog Title',
        description='Generates a blog title based on a topic',
        tags=['title'],
        examples=[
            'Can you give me a title for this article?',
        ],
    ),
]
```

10. Find the comment **Create agent card** and add this code to define the metadata that makes the agent discoverable:

Code Copy

```
# Create agent card
agent_card = AgentCard(
    name='Microsoft Foundry Title Agent',
    description='An intelligent title generator agent powered by Foundry. '
    'I can help you generate catchy titles for your articles.',
    url=f'http://{host}:{port}/',
    version='1.0.0',
    default_input_modes=['text'],
    default_output_modes=['text'],
    capabilities=AgentCapabilities(),
    skills=skills,
)
```

11. Locate the comment **Create agent executor** and add the following code to initialize the agent executor using the agent card:

Code Copy

```
# Create agent executor
agent_executor = create_foundry_agent_executor(agent_card)
```

The agent executor will act as a wrapper for the title agent you created.

12. Find the comment **Create request handler** and add the following to handle incoming requests using the executor:

Code Copy

```
# Create request handler
request_handler = DefaultRequestHandler(
    agent_executor=agent_executor, task_store=InMemoryTaskStore()
)
```

13. Under the comment **Create A2A application**, add this code to create the A2A-compatible application instance:

Code Copy

```
# Create A2A application
a2a_app = A2AStarletteApplication(
    agent_card=agent_card, http_handler=request_handler
)
```

This code creates an A2A server that will share the title agent's information and handle incoming requests for this agent using the title agent executor.

14. Save the code file (*CTRL+S*) when you have finished.

Enable messages between the agents

In this task, you use the A2A protocol to enable the routing agent to send messages to the other agents. You also allow the title agent to receive messages by implementing the agent executor class.

1. Open the **routing_agent/agent.py** file in the code editor.

   The routing agent acts as an orchestrator that handles user messages and determines which remote agent should process the request.

   When a user message is received, the routing agent:

   - Starts a conversation thread.
   - Uses the `create_and_process` method to evaluate the best-matching agent for the user's message.
   - The message is routed to the appropriate agent over HTTP using the `send_message` function.
   - The remote agent processes the message and returns a response.

   The routing agent finally captures the response and returns it to the user through the thread.

   Notice that the `send_message` method is async and must be awaited for the agent run to complete successfully.

2. Add the following code under the comment **Retrieve the remote agent's A2A client using the agent name**:

Code Copy

```
# Retrieve the remote agent's A2A client using the agent name
client = self.remote_agent_connections[agent_name]
```

3. Locate the comment **Construct the payload to send to the remote agent** and add the following code:

Code Copy

```
# Construct the payload to send to the remote agent
payload: dict[str, Any] = {
    'message': {
        'role': 'user',
        'parts': [{'kind': 'text', 'text': task}],
        'messageId': message_id,
    },
}
```

4. Find the comment **Wrap the payload in a SendMessageRequest object** and add the following code:

Code Copy

```
# Wrap the payload in a SendMessageRequest object
message_request = SendMessageRequest(id=message_id,
params=MessageSendParams.model_validate(payload))
```

5. Add the following code under the comment **Send the message to the remote agent client and await the response**:

Code Copy

```
# Send the message to the remote agent client and await the response
send_response: SendMessageResponse = await
client.send_message(message_request=message_request)
```

6. Save the code file (*CTRL+S*) when you have finished. Now the routing agent is able to discover and send messages to the title agent. Let's create the agent executor code to handle those incoming messages from the routing agent.

7. Open the **title_agent/agent_executor.py** file in the code editor.

   The `AgentExecutor` class implementation must contain the methods `execute` and `cancel`. The cancel method has been provided for you. The `execute` method includes a `TaskUpdater` object that manages events and signals to the caller when the task is complete. Let's add the logic for task execution.

8. In the `execute` method, add the following code under the comment **Process the request**:

Code Copy

```
# Process the request
await self._process_request(context.message.parts, context.context_id, updater)
```

9. In the `_process_request` method, add the following code under the comment **Get the title agent**:

Code Copy

```
# Get the title agent
agent = await self._get_or_create_agent()
```

10. Add the following code under the comment **Update the task status**:

Code Copy

```
# Update the task status
await task_updater.update_status(
    TaskState.working,
    message=new_agent_text_message('Title Agent is processing your request...',
context_id=context_id),
)
```

11. Find the comment **Run the agent conversation** and add the following code:

Code Copy

```
# Run the agent conversation
responses = await agent.run_conversation(user_message)
```

12. Find the comment **Update the task with the responses** and add the following code:

Code Copy

```
# Update the task with the responses
for response in responses:
    await task_updater.update_status(
        TaskState.working,
        message=new_agent_text_message(response, context_id=context_id),
    )
```

13. Find the comment **Mark the task as complete** and add the following code:

Code Copy

```
# Mark the task as complete
final_message = responses[-1] if responses else 'Task completed.'
await task_updater.complete(
    message=new_agent_text_message(final_message, context_id=context_id)
)
```

Now your title agent has been wrapped with an agent executor that the A2A protocol will use to handle messages. Great work!

Test the application

1. In the integrated terminal, enter the following commands to run the application:

Code Copy

```
az login
```

Code Copy

```
python run_all.py
```

The application runs using the credentials for your authenticated Azure session to connect to your project and create and run the agent. You should see some output from each server as it starts.

2. Wait until the prompt for input appears, then enter a prompt such as:

Code Copy

```
Create a title and outline for an article about React programming.
```

After a few moments, you should see a response from the agent with the results.

3. Enter `quit` to exit the program and stop the servers.

   You can also use `deactivate` to exit the Python virtual environment in the terminal.

Clean up

If you've finished exploring Azure AI Agent Service, you should delete the resources you have created in this exercise to avoid incurring unnecessary Azure costs.

1. Return to the browser tab containing the Azure portal (or re-open the Azure portal at `https://portal.azure.com` in a new browser tab) and view the contents of the resource group where you deployed the resources used in this exercise.
2. On the toolbar, select **Delete resource group**.
3. Enter the resource group name and confirm that you want to delete it.